TRAVAUX PRATIQUES

Pipeline Big Data CI/CD

Hadoop HDFS | Spark | MLlib | Git | Docker | GitHub Actions | Azure / SSH

Niveau	TS - Big Data & Data Engineering
Prerequis	Python, Linux, Git, notions Docker, HDFS
Stockage	Hadoop HDFS (local Docker ou Azure HDInsight)
Deploiement	Serveur SSH ou Azure (HDInsight / Databricks)
Dataset	NYC Taxi Trips — TLC / NYC OpenData

Sommaire

Partie 1 — Presentation et contexte 3
Partie 2 — Donnees NYC Taxi Trips + HDFS 4
Partie 3 — Gestion de version avec Git 7
Partie 4 — Environnement Hadoop HDFS + Spark 9
Partie 5 — Pipeline Spark / MLlib avec HDFS 12
Partie 6 — Tests unitaires pytest 16
Partie 7 — Containerisation Docker + HDFS 18
Partie 8 — CI GitHub Actions 20
Partie 9 — CD — Deploiement SSH ou Azure 23
Partie 10 — Monitoring et synthese 28

Partie 1 — Presentation et Contexte

1.1 Objectifs pedagogiques

Ce TP construit un pipeline Big Data complet de bout en bout avec une infrastructure realiste :

- **HDFS** : stocker les donnees brutes et transformees dans Hadoop Distributed File System
- **Spark** : traiter et transformer les donnees avec PySpark
- **MLlib** : predire la duree des trajets avec un modele distribue
- **Git** : gerer le code source avec branches et commits semantiques
- **Docker** : reproduire un cluster Hadoop+Spark en local
- **GitHub Actions** : automatiser tests, build et deploiement (CI/CD)
- **SSH ou Azure** : deployer en production sur un serveur reel ou dans le cloud

1.2 Architecture globale

Architecture cible — pipeline de bout en bout

```
[Donnees brutes] ──► HDFS /data/raw/        (stockage distribue)
      |
      ▼
[PySpark Job]    ──► Lecture HDFS + nettoyage + feature engineering
      |
      ▼
[MLlib Pipeline] ──► Entrainement RandomForest + evaluation metriques
      |
      ▼
[HDFS /data/processed/] ──► Predictions sauvegardees en Parquet
      |
[CI/CD]  ──► lint ► tests ► Docker build ► push Docker Hub
      |
[CD]     ──► SSH sur VM Linux  OU  Azure HDInsight / Databricks
```

Partie 2 — Donnees NYC Taxi + Stockage HDFS

2.1 Source des donnees

Le dataset utilise est le NYC Taxi and Limousine Commission (TLC) Trip Record Data, un jeu de donnees Big Data reel, mis a jour mensuellement par la ville de New York.

Source officielle des donnees

URL : https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
CDN Parquet : https://d37ci6vzurychx.cloudfront.net/trip-data/
Format : Parquet (recommande) ou CSV
Volume : ~500 MB / mois en Parquet | ~2 GB / mois en CSV
Licence : NYC Open Data — libre d'utilisation

Fichiers a telecharger pour ce TP (Yellow Taxi 2023) :
- yellow_tripdata_2023-01.parquet
- yellow_tripdata_2023-02.parquet
- yellow_tripdata_2023-03.parquet

2.2 Schema des donnees

Colonne	Type	Description
VendorID	Integer	Fournisseur (1=CMT, 2=VTS)
tpep_pickup_datetime	Timestamp	Date/heure de prise en charge
tpep_dropoff_datetime	Timestamp	Date/heure de depose
passenger_count	Integer	Nombre de passagers
trip_distance	Float	Distance parcourue en miles
PULocationID	Integer	Zone de prise en charge
DOLocationID	Integer	Zone de depose
fare_amount	Float	Montant de la course (USD)
tip_amount	Float	Pourboire (USD)
total_amount	Float	Montant total paye (USD)
trip_duration_min (*)	Float	VARIABLE CIBLE — calculee (min)

2.3 Architecture HDFS — Organisation des dossiers

Structure HDFS du projet

Le cluster HDFS organise les donnees en zones distinctes :

```
hdfs://namenode:9000/
├── data/
│   ├── raw/                # Donnees brutes Parquet (lecture seule)
│   │   ├── yellow_tripdata_2023-01.parquet
│   │   ├── yellow_tripdata_2023-02.parquet
│   │   └── yellow_tripdata_2023-03.parquet
│   ├── processed/          # Donnees nettoyees et features engineerees
│   │   └── features/
│   └── predictions/        # Predictions du modele MLlib
│       └── year=2023/month=01-03/
├── models/                 # Modeles MLlib sauvegardes
│   └── rf_model_v1/
└── logs/                   # Logs Spark History Server
```

2.4 Telechargement et ingestion dans HDFS

Suivez les etapes suivantes pour charger les donnees dans HDFS :

```
# ETAPE 1 — Telecharger les fichiers Parquet en local
mkdir -p /tmp/taxi-data && cd /tmp/taxi-data
BASE=https://d37ci6vzurychx.cloudfront.net/trip-data
for m in 01 02 03; do
  wget ${BASE}/yellow_tripdata_2023-${m}.parquet
done

# ETAPE 2 — Creer la structure de dossiers dans HDFS
hdfs dfs -mkdir -p /data/raw
hdfs dfs -mkdir -p /data/processed/features
hdfs dfs -mkdir -p /data/predictions
hdfs dfs -mkdir -p /models

# ETAPE 3 — Uploader les fichiers vers HDFS
hdfs dfs -put /tmp/taxi-data/*.parquet /data/raw/

# ETAPE 4 — Verifier l'upload
hdfs dfs -ls -h /data/raw/
hdfs dfs -du -s -h /data/raw/

# ETAPE 5 — Verifier la replication (facteur 3 par defaut)
hdfs fsck /data/raw/ -files -blocks -locations
```

2.5 Questions — Donnees et HDFS

Question 2.1 — Ingestion dans HDFS

a) Quelle commande permet de verifier le facteur de replication d'un fichier dans HDFS ?

b) Changez le facteur de replication du dossier /data/raw/ a 2. Quelle commande utilisez-vous ?

c) Quelle est la difference entre 'hdfs dfs -put' et 'hdfs dfs -copyFromLocal' ?

d) Expliquez le concept de NameNode et DataNode dans l'architecture HDFS. Que se passe-t-il si le NameNode tombe ?

Question 2.2 — Gestion HDFS

a) Listez les commandes HDFS equivalentes a : ls, mkdir, rm, cat, mv, cp.

b) Quelle commande affiche l'espace disque utilise et disponible dans HDFS ?

c) Expliquez le role du HDFS Web UI (port 9870). Quelles informations y trouve-t-on ?

d) Pourquoi le stockage HDFS est-il preferable au stockage local pour un cluster Spark multi-noeuds ?

Partie 3 — Gestion de version avec Git

3.1 Initialisation et structure

```
# Initialiser le depot Git
mkdir nyc-taxi-bigdata-cicd && cd nyc-taxi-bigdata-cicd
git init
git config user.name  'Votre Nom'
git config user.email 'votre@email.com'

# Structure du projet
mkdir -p .github/workflows src jobs tests docker notebooks
mkdir -p data/raw data/processed   # dossiers locaux (gitignore)
touch src/__init__.py tests/__init__.py

# .gitignore adapte Big Data + HDFS
cat > .gitignore << 'EOF'
# Donnees (stockees dans HDFS, pas dans Git)
data/raw/
data/processed/
*.parquet
*.csv
# Modeles ML (lourds, versionnes dans HDFS ou MLflow)
models/
*.pkl
# Python
__pycache__/
*.pyc
.pytest_cache/
htmlcov/
.coverage
# Environnements
.env
venv/
# Jupyter
.ipynb_checkpoints/
EOF

git add .
git commit -m 'feat: initialisation structure projet Big Data HDFS CI/CD'
```

3.2 Strategie de branches

Convention Git Flow simplifie

main --> production (protegee — PR + CI obligatoires)
develop --> integration (merge des features validees)
feature/X --> nouvelle fonctionnalite (ex: feature/hdfs-ingestion)
fix/X --> correction de bug
hotfix/X --> urgence production

3.3 Questions — Git

Question 3.1 — Premiers commits et gestion des branches

a) Differenciez 'git add .' et 'git add -p'. Dans quel cas privilegier chacun ?

b) Creez les branches feature/hdfs-ingestion et feature/preprocessing, mergez-les dans develop.

c) Simulez un conflit de merge sur src/preprocessing.py entre deux branches. Resolvez-le.

d) Differenciez git merge et git rebase. Quand utiliser l'un ou l'autre dans ce projet ?

Question 3.2 — Historique, tags et outils avances

a) Creez un tag v1.0.0 annote sur le premier commit de pipeline fonctionnel. Poussez-le sur GitHub.

b) Expliquez git bisect et comment l'utiliser pour trouver le commit qui a casse les tests HDFS.

c) Qu'est-ce que git stash ? Donnez un cas d'usage precis dans ce projet.

d) Quelle est la difference entre .gitignore et .gitkeep ? Pourquoi mettre un .gitkeep dans data/raw/ ?

Partie 4 — Environnement Hadoop HDFS + Spark

4.1 Configuration Hadoop (core-site.xml)

Le fichier core-site.xml configure l'adresse du NameNode HDFS. Il est monte dans le container Docker.

```xml
<!-- docker/hadoop/core-site.xml -->
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://namenode:9000</value>
  </property>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>
  <property>
    <name>dfs.blocksize</name>
    <value>134217728</value>  <!-- 128 MB -->
  </property>
</configuration>
```

4.2 SparkSession configuree pour HDFS

```python
# src/spark_session.py
import os
from pyspark.sql import SparkSession

HDFS_NAMENODE = os.getenv('HDFS_NAMENODE', 'hdfs://namenode:9000')

def get_spark(app_name: str = 'NYC-Taxi') -> SparkSession:
    return (
        SparkSession.builder
        .appName(app_name)
        # Connexion HDFS
        .config('spark.hadoop.fs.defaultFS', HDFS_NAMENODE)
        # Optimisations Spark
        .config('spark.sql.adaptive.enabled', 'true')
        .config('spark.sql.shuffle.partitions', '8')
        .config('spark.driver.memory', '2g')
        .config('spark.executor.memory', '2g')
        # Spark History Server
        .config('spark.eventLog.enabled', 'true')
        .config('spark.eventLog.dir', f'{HDFS_NAMENODE}/logs/spark-events')
        .getOrCreate()
    )
```

4.3 Questions — HDFS et Spark

Question 4.1 — Integration Spark + HDFS

a) Ecrivez un script verify_hdfs.py qui : lance une SparkSession HDFS, liste /data/raw/ via spark.read, affiche le nombre de fichiers et la taille totale.

b) Pourquoi utilise-t-on hdfs://namenode:9000 et non file:// pour lire les donnees en cluster ?

c) Quelle est la taille d'un block HDFS par defaut (128 MB) ? Pourquoi ce choix par rapport au Block NTFS (4 KB) ?

d) Quel est le role de spark.eventLog.dir pointe vers HDFS ? Comment consulter le Spark History Server ?

Partie 5 — Pipeline Spark / MLlib avec HDFS

5.1 Module d'ingestion HDFS (src/ingestion.py)

```
# src/ingestion.py
from pyspark.sql import SparkSession, DataFrame
import logging

logger = logging.getLogger(__name__)

EXPECTED_COLUMNS = [
    'VendorID', 'tpep_pickup_datetime', 'tpep_dropoff_datetime',
    'passenger_count', 'trip_distance', 'fare_amount', 'total_amount'
]

def load_taxi_data(spark: SparkSession, hdfs_path: str) -> DataFrame:
    """Charge les fichiers Parquet depuis HDFS et valide le schema."""
    logger.info(f'[HDFS] Lecture : {hdfs_path}')
    df = spark.read.parquet(hdfs_path)
    missing = [c for c in EXPECTED_COLUMNS if c not in df.columns]
    if missing:
        raise ValueError(f'Colonnes manquantes : {missing}')
    logger.info(f'[HDFS] {df.count():,} lignes | {len(df.columns)} colonnes')
    return df

def save_to_hdfs(df: DataFrame, hdfs_path: str, mode='overwrite') -> None:
    """Sauvegarde un DataFrame en Parquet partitionne dans HDFS."""
    logger.info(f'[HDFS] Ecriture -> {hdfs_path}')
    (df.write
       .mode(mode)
       .partitionBy('year', 'month')
       .parquet(hdfs_path))
    logger.info(f'[HDFS] Sauvegarde terminee.')
```

5.2 Module preprocessing (src/preprocessing.py)

```
# src/preprocessing.py
from pyspark.sql import DataFrame
from pyspark.sql import functions as F

def clean_data(df: DataFrame) -> DataFrame:
    return df.filter(
        (F.col('trip_distance')   >  0.1)  & (F.col('trip_distance')   < 100)  &
        (F.col('fare_amount')     >  2.5)  & (F.col('fare_amount')     < 500)  &
        (F.col('passenger_count') >= 1)    & (F.col('passenger_count') <= 6)   &
        F.col('tpep_pickup_datetime').isNotNull()  &
        F.col('tpep_dropoff_datetime').isNotNull()
    )

def engineer_features(df: DataFrame) -> DataFrame:
    return (
        df
        .withColumn('trip_duration_min',
            (F.unix_timestamp('tpep_dropoff_datetime') -
             F.unix_timestamp('tpep_pickup_datetime')) / 60.0)
        .withColumn('hour_of_day',  F.hour('tpep_pickup_datetime'))
        .withColumn('day_of_week',  F.dayofweek('tpep_pickup_datetime'))
        .withColumn('is_weekend',
            F.dayofweek('tpep_pickup_datetime').isin([1,7]).cast('integer'))
```

```
        .withColumn('is_rush_hour',
            F.when(F.hour('tpep_pickup_datetime').between(7,9)  |
                   F.hour('tpep_pickup_datetime').between(17,19), 1).otherwise(0))
        .withColumn('speed_mph',
            F.when(F.col('trip_duration_min')>0,
                F.col('trip_distance')/(F.col('trip_duration_min')/60.0)).otherwise(None))
        .withColumn('fare_per_mile',
            F.when(F.col('trip_distance')>0,
                F.col('fare_amount')/F.col('trip_distance')).otherwise(None))
        .withColumn('year',  F.year('tpep_pickup_datetime').cast('string'))
        .withColumn('month', F.month('tpep_pickup_datetime').cast('string'))
        .filter(
            (F.col('trip_duration_min') > 1) &
            (F.col('trip_duration_min') < 180) &
            (F.col('speed_mph') < 80))
    )
```

5.3 Job principal HDFS (jobs/run_pipeline.py)

```
# jobs/run_pipeline.py
import argparse
from src.spark_session import get_spark
from src.ingestion    import load_taxi_data, save_to_hdfs
from src.preprocessing import clean_data, engineer_features
from src.model         import train_and_evaluate

def run(hdfs_input: str, hdfs_output: str, hdfs_models: str):
    spark = get_spark('NYC-Taxi-Pipeline')

    # Lecture depuis HDFS
    df_raw = load_taxi_data(spark, hdfs_input)
    df_raw.cache()

    # Traitement
    df_clean    = clean_data(df_raw)
    df_featured = engineer_features(df_clean)
    df_featured.repartition(8).cache()

    # Sauvegarde features dans HDFS (partitionnees par annee/mois)
    save_to_hdfs(df_featured, f'{hdfs_output}/features')

    # Entrainement MLlib
    results = train_and_evaluate(df_featured)
    print(f"RF  RMSE={results['rf']['rmse']:.2f} R2={results['rf']['r2']:.3f}")

    # Sauvegarder predictions dans HDFS
    _, test = df_featured.randomSplit([0.8, 0.2], seed=42)
    preds = results['rf']['model'].transform(test)
    save_to_hdfs(preds.select('trip_duration_min','prediction','year','month'),
                 f'{hdfs_output}/predictions')

    # Sauvegarder le modele dans HDFS
    results['rf']['model'].write().overwrite().save(f'{hdfs_models}/rf_model_v1')

    spark.stop()

if __name__ == '__main__':
    p = argparse.ArgumentParser()
    p.add_argument('--input',  default='hdfs://namenode:9000/data/raw')
    p.add_argument('--output', default='hdfs://namenode:9000/data/processed')
    p.add_argument('--models', default='hdfs://namenode:9000/models')
    a = p.parse_args()
    run(a.input, a.output, a.models)
```

5.4 Questions — Pipeline HDFS / MLlib

Question 5.1 — Lecture et exploration depuis HDFS

a) Lisez les 3 fichiers Parquet en une seule commande spark.read.parquet() avec wildcard HDFS.

b) Affichez le plan d'execution (df.explain(True)) apres un filter. Identifiez le predicate pushdown Parquet.

c) Combien de partitions Spark le DataFrame a-t-il apres la lecture HDFS ? Pourquoi ce nombre ?

d) Apres engineer_features, sauvegardez le DataFrame partitionne par 'year' et 'month' dans HDFS.

Question 5.2 — MLlib et sauvegarde HDFS

a) Obtenez RMSE, MAE et R2. Comparez RandomForestRegressor et LinearRegression.

b) Sauvegardez le meilleur modele dans HDFS (hdfs://namenode:9000/models/rf_v1).

c) Rechargez le modele depuis HDFS avec PipelineModel.load() et appliquez-le sur de nouvelles donnees.

d) Affichez les feature importances et interpretez les 3 plus importantes.

Partie 6 — Tests unitaires avec pytest

6.1 Configuration (tests/conftest.py)

```
# tests/conftest.py
import pytest, random
from datetime import datetime, timedelta
from pyspark.sql import SparkSession

@pytest.fixture(scope='session')
def spark():
    session = (SparkSession.builder
        .master('local[2]')
        .appName('NYC-Tests')
        # Mode local : pas de HDFS reel, on utilise file://
        .config('spark.sql.shuffle.partitions', '4')
        .config('spark.driver.memory', '2g')
        .getOrCreate())
    session.sparkContext.setLogLevel('ERROR')
    yield session
    session.stop()

@pytest.fixture(scope='session')
def sample_df(spark):
    random.seed(42)
    base = datetime(2023, 1, 15, 8, 0, 0)
    data = [(
        1, base+timedelta(minutes=i*30),
        base+timedelta(minutes=i*30+random.randint(5,45)),
        random.randint(1,4), round(random.uniform(0.5,20),2),
        1, 'N', random.randint(1,263), random.randint(1,263),
        1, round(random.uniform(5,50),2),
        0.5, 0.5, round(random.uniform(0,10),2), 0.0,
        round(random.uniform(10,70),2)
    ) for i in range(60)]
    cols=['VendorID','tpep_pickup_datetime','tpep_dropoff_datetime',
          'passenger_count','trip_distance','RatecodeID','store_and_fwd_flag',
          'PULocationID','DOLocationID','payment_type','fare_amount',
          'extra','mta_tax','tip_amount','tolls_amount','total_amount']
    return spark.createDataFrame(data, cols)
```

6.2 Questions — Tests

Question 6.1 — Tests d'ingestion et de preprocessing

a) Ecrivez 4 tests dans test_ingestion.py : retour DataFrame, colonnes attendues, exception fichier absent, ValueError schema invalide.

b) Ecrivez 3 tests dans test_preprocessing.py : filtre distance negative, duree positive, plage hour_of_day 0-23.

c) Ecrivez 3 tests dans test_model.py : type Pipeline, colonne 'prediction' presente, R2 > 0.

d) Obtenez un taux de couverture >= 80% et joignez le rapport HTML au livrable.

Question 6.2 — Simulation HDFS dans les tests

a) Dans les tests, comment simuler la lecture HDFS sans cluster reel ? (tmp_path + file://).

b) Ecrivez un test qui sauvegarde un DataFrame en Parquet local (tmp_path), le relit et verifie le count.

c) Expliquez pourquoi on evite d'utiliser un vrai NameNode HDFS dans les tests unitaires.

Partie 7 — Containerisation Docker + HDFS

7.1 docker-compose.yml avec cluster HDFS complet

```yaml
# docker/docker-compose.yml
version: '3.9'

services:
  # ── HDFS NameNode ──────────────────────────────────────
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    hostname: namenode
    environment:
      - CLUSTER_NAME=taxi-hdfs
      - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
    ports:
      - '9870:9870'   # HDFS Web UI
      - '9000:9000'   # HDFS RPC
    volumes:
      - hadoop_namenode:/hadoop/dfs/name
      - ./hadoop/core-site.xml:/etc/hadoop/core-site.xml
    networks: [bigdata]

  # ── HDFS DataNode 1 ────────────────────────────────────
  datanode1:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode1
    environment:
      - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
    volumes:
      - hadoop_datanode1:/hadoop/dfs/data
    depends_on: [namenode]
    networks: [bigdata]

  # ── HDFS DataNode 2 ────────────────────────────────────
  datanode2:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode2
    environment:
      - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
    volumes:
      - hadoop_datanode2:/hadoop/dfs/data
    depends_on: [namenode]
    networks: [bigdata]

  # ── Spark Master ───────────────────────────────────────
  spark-master:
    image: bitnami/spark:3.5.0
    container_name: spark-master
    environment:
      - SPARK_MODE=master
      - SPARK_MASTER_PORT=7077
      - HADOOP_CONF_DIR=/etc/hadoop
    ports:
      - '8080:8080'   # Spark Master UI
      - '7077:7077'
    volumes:
      - ./hadoop/core-site.xml:/etc/hadoop/core-site.xml
    depends_on: [namenode]
    networks: [bigdata]
```

```yaml
  # ── Spark Worker ──────────────────────────────────────
  spark-worker:
    image: bitnami/spark:3.5.0
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
      - SPARK_WORKER_MEMORY=2G
      - SPARK_WORKER_CORES=2
      - HADOOP_CONF_DIR=/etc/hadoop
    volumes:
      - ./hadoop/core-site.xml:/etc/hadoop/core-site.xml
    depends_on: [spark-master, namenode]
    networks: [bigdata]

  # ── Jupyter + PySpark ─────────────────────────────────
  jupyter:
    build: { context: .., dockerfile: docker/Dockerfile }
    container_name: jupyter-spark
    environment:
      - SPARK_MASTER=spark://spark-master:7077
      - HDFS_NAMENODE=hdfs://namenode:9000
    ports:
      - '8888:8888'   # Jupyter Lab
      - '4040:4040'   # Spark App UI
    volumes:
      - ./hadoop/core-site.xml:/etc/hadoop/core-site.xml
      - ../notebooks:/app/notebooks
    depends_on: [spark-master, namenode]
    networks: [bigdata]
    command: jupyter lab --ip=0.0.0.0 --no-browser --allow-root

volumes:
  hadoop_namenode:
  hadoop_datanode1:
  hadoop_datanode2:

networks:
  bigdata:
    driver: bridge
```

7.2 Questions — Docker + HDFS

Question 7.1 — Demarrage et verification du cluster

a) Demarrez le cluster avec 'docker-compose up -d'. Verifiez que le NameNode et les 2 DataNodes sont actifs dans HDFS Web UI (localhost:9870).

b) Combien de blocs HDFS ont ete crees apres l'upload des 3 fichiers Parquet ? Calculez theoriquement.

c) Expliquez pourquoi core-site.xml est monte en volume dans chaque container. Que se passe-t-il sans ce fichier ?

d) Arretez un DataNode (docker stop datanode1). Le cluster reste-t-il fonctionnel ? Expliquez avec le facteur de replication.

Partie 8 — Pipeline CI avec GitHub Actions

8.1 Workflow CI (.github/workflows/ci.yml)

```yaml
name: CI — Big Data Pipeline HDFS

on:
  push:
    branches: [ main, develop, 'feature/**' ]
  pull_request:
    branches: [ main, develop ]

env:
  PYTHON_VERSION: '3.10'
  JAVA_VERSION:   '11'
  PYSPARK_PYTHON: python3

jobs:
  lint:
    name: Lint and Format
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '${{ env.PYTHON_VERSION }}', cache: pip }
      - run: pip install flake8 black isort
      - run: black  --check --diff src/ tests/ jobs/
      - run: isort  --check-only --diff src/ tests/ jobs/
      - run: flake8 src/ tests/ jobs/ --max-line-length=100

  test:
    name: Tests Spark (mode local — sans HDFS reel)
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '${{ env.JAVA_VERSION }}' }
      - uses: actions/setup-python@v5
        with: { python-version: '${{ env.PYTHON_VERSION }}', cache: pip }
      - uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - name: Run tests
        env:
          PYSPARK_PYTHON: python3
          # En CI, pas de HDFS : les tests utilisent file:// via tmp_path
          HDFS_NAMENODE: 'file://'
        run: |
          pytest tests/ --cov=src --cov-report=xml
                        --cov-report=html --cov-fail-under=80 -v
      - uses: codecov/codecov-action@v4
        with: { file: coverage.xml }
      - uses: actions/upload-artifact@v4
        with: { name: coverage-report, path: htmlcov/, retention-days: 7 }

  build:
    name: Build Docker Image
    runs-on: ubuntu-latest
```

```
    needs: test
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}
      - uses: docker/setup-buildx-action@v3
      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ secrets.DOCKER_USERNAME }}/nyc-taxi-spark
          tags: |
            type=sha
            type=ref,event=branch
      - uses: docker/build-push-action@v5
        with:
          context: .
          file: docker/Dockerfile
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

8.2 Questions — CI

Question 8.1 — Analyse et comprehension

a) Pourquoi les tests CI utilisent HDFS_NAMENODE='file://' et non le vrai cluster HDFS Docker ?

b) Expliquez le mot-cle 'needs'. Que se passe-t-il si le job lint echoue ?

c) Pourquoi active-t-on le cache pip (actions/cache) ? Mesurez le gain de temps entre un run froid et un run chaud.

d) Expliquez la condition 'push: github.event_name != pull_request'. Pourquoi ne pas pousser l'image depuis une PR ?

Partie 9 — CD : Deploiement SSH ou Azure

9.1 Vue d'ensemble — deux options de deploiement

Deux strategies de deploiement en production

Option A — Serveur SSH (VM Linux) :
Machine virtuelle (VPS, serveur on-premise, VM cloud) avec Hadoop + Spark installe.
Acces via cle SSH. Recommande pour : apprentissage, budget reduit, controle total.

Option B — Azure HDInsight ou Databricks :
Service manage cloud Microsoft. Cluster Hadoop/Spark provisionne automatiquement.
Acces via API REST ou CLI Azure. Recommande pour : production, scalabilite, equipes.

9.2 Option A — Deploiement via SSH (serveur Linux)

9.2.1 Configuration du serveur cible

Prerequis du serveur SSH

OS : Ubuntu 22.04 LTS (ou Debian 11)
RAM : minimum 8 GB (recommande 16 GB pour Spark + HDFS)
CPU : minimum 4 vCPU
Disque : minimum 100 GB pour les donnees HDFS
Ports : 22 (SSH), 8080 (Spark UI), 9870 (HDFS UI), 4040 (Spark App)

Logiciels a installer sur le serveur :
sudo apt update && sudo apt install -y openjdk-11-jdk docker.io docker-compose git
sudo systemctl enable --now docker
sudo usermod -aG docker $USER

9.2.2 Generation de la cle SSH dediee au deploiement

```
# ─── Sur votre machine locale ──────────────────────────────────

# Generer une paire de cles SSH ED25519 dediee au deploiement
# (NE JAMAIS reutiliser votre cle personnelle ~/.ssh/id_ed25519)
ssh-keygen -t ed25519 \
           -C 'github-actions-deploy-nyc-taxi' \
           -f ~/.ssh/deploy_nyc_taxi \
           -N ''   # pas de passphrase (automatisation CI/CD)

# Afficher la cle PUBLIQUE -> a copier sur le serveur
cat ~/.ssh/deploy_nyc_taxi.pub

# Afficher la cle PRIVEE -> a ajouter dans GitHub Secrets
cat ~/.ssh/deploy_nyc_taxi
```

```
# ─── Sur le serveur distant ──────────────────────────────────────

# Creer le dossier .ssh si absent
mkdir -p ~/.ssh && chmod 700 ~/.ssh

# Ajouter la cle PUBLIQUE dans authorized_keys
echo '<coller_ici_le_contenu_de_deploy_nyc_taxi.pub>' >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# Tester la connexion SSH depuis votre machine locale
ssh -i ~/.ssh/deploy_nyc_taxi ubuntu@<IP_SERVEUR> 'echo Connexion OK'
```

9.2.3 Secrets GitHub a configurer pour SSH

Secret GitHub	Valeur / Description	Ou le creer
SERVER_HOST	IP publique ou nom de domaine du serveur	Ex: 51.103.42.17 ou monserveur.com
SERVER_USER	Utilisateur Linux SSH	Ex: ubuntu, debian, ec2-user
SERVER_PORT	Port SSH (defaut 22)	Changer si securite renforcee (ex: 2222)
SSH_PRIVATE_KEY	Contenu complet de deploy_nyc_taxi	Copier/coller tout le fichier (BEGIN...END)
DOCKER_USERNAME	Username Docker Hub	hub.docker.com > Account Settings
DOCKER_PASSWORD	Token Docker Hub	Account Settings > Security > New Token
HDFS_NAMENODE_SSH	URL HDFS du serveur	hdfs://51.103.42.17:9000

9.2.4 Workflow CD — Option SSH

```
# .github/workflows/cd-ssh.yml
name: CD — Deploiement SSH + HDFS

on:
  push:
    branches: [ main ]

jobs:
  deploy-ssh:
    name: Deploy via SSH
    runs-on: ubuntu-latest
    environment:
      name: production-ssh
      url: http://${{ secrets.SERVER_HOST }}:8080

    steps:
      - uses: actions/checkout@v4

      - name: Deploy and run via SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host:     ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          port:     ${{ secrets.SERVER_PORT }}
          key:      ${{ secrets.SSH_PRIVATE_KEY }}
          script: |
            set -e
```

```
            echo '=== Mise a jour du code ==='
            cd /opt/nyc-taxi-bigdata
            git pull origin main

            echo '=== Pull de la nouvelle image Docker ==='
            docker pull ${{ secrets.DOCKER_USERNAME }}/nyc-taxi-spark:main

            echo '=== Redemarrage cluster Hadoop + Spark ==='
            docker-compose -f docker/docker-compose.yml down
            docker-compose -f docker/docker-compose.yml up -d
            sleep 40  # attendre que HDFS soit pret

            echo '=== Verifier HDFS ==='
            docker exec namenode hdfs dfs -ls /data/raw/ || \
              (docker exec namenode hdfs dfs -mkdir -p /data/raw && echo 'Dossier cree')

            echo '=== Soumission du job Spark vers HDFS ==='
            docker exec jupyter-spark spark-submit \
              --master spark://spark-master:7077 \
              --deploy-mode client \
              --driver-memory 2g \
              --executor-memory 2g \
              --num-executors 2 \
              --conf spark.hadoop.fs.defaultFS=${{ secrets.HDFS_NAMENODE_SSH }} \
              /app/jobs/run_pipeline.py \
              --input  ${{ secrets.HDFS_NAMENODE_SSH }}/data/raw \
              --output ${{ secrets.HDFS_NAMENODE_SSH }}/data/processed \
              --models ${{ secrets.HDFS_NAMENODE_SSH }}/models

      - name: Health Check Spark UI
        run: |
          sleep 15
          curl -f http://${{ secrets.SERVER_HOST }}:8080 || exit 1
          echo 'Spark UI accessible'

      - name: Health Check HDFS
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key:  ${{ secrets.SSH_PRIVATE_KEY }}
          script: |
            docker exec namenode hdfs dfsadmin -report | grep 'Live datanodes'
```

9.3 Option B — Deploiement Azure HDInsight / Databricks

9.3.1 Presentation des services Azure

Services Azure disponibles pour ce projet

HDInsight : cluster Hadoop/Spark manage par Azure.
  -> Inclut HDFS (Azure Blob Storage via WASB ou ADLS Gen2).
  -> Provisionne en ~15 min, facture a l'heure.
  -> Acces via SSH ou REST API Livy (soumission de jobs).

Databricks : plateforme Spark optimisee par Microsoft + Databricks.
  -> Cluster Spark auto-scaling, notebooks collaboratifs.
  -> Stockage : Azure Data Lake Storage Gen2 (ADLS2) ou Azure Blob Storage.

-> Acces via CLI databricks ou REST API.

Azure Blob Storage / ADLS Gen2 : equivalent cloud de HDFS.
-> URI : wasbs://container@account.blob.core.windows.net/
-> Ou : abfss://container@account.dfs.core.windows.net/

9.3.2 Secrets GitHub a configurer pour Azure

Secret GitHub	Valeur / Description	Ou le trouver dans Azure
AZURE_CREDENTIALS	JSON Service Principal complet	az ad sp create-for-rbac --sdk-auth
AZURE_SUBSCRIPTION_ID	ID abonnement Azure	Portal Azure > Abonnements
AZURE_RESOURCE_GROUP	Nom du groupe de ressources	Portal Azure > Groupes de ressources
HDINSIGHT_CLUSTER_NAME	Nom du cluster HDInsight	Portal Azure > HDInsight clusters
HDINSIGHT_SSH_USER	Utilisateur SSH HDInsight	Defaut : sshuser
HDINSIGHT_SSH_KEY	Cle SSH privee pour HDInsight	Generee a la creation du cluster
AZURE_STORAGE_ACCOUNT	Nom du compte de stockage ADLS2	Portal Azure > Storage accounts
AZURE_STORAGE_KEY	Cle d'acces au stockage	Storage > Access keys > key1
DATABRICKS_HOST	URL workspace Databricks	Ex: https://adb-xxx.azuredatabricks.net
DATABRICKS_TOKEN	Token API Databricks	Databricks > User Settings > Access Tokens

9.3.3 Provisionner Azure HDInsight via Azure CLI

```
# ─── Installer Azure CLI ──────────────────────────────────
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az login

# ─── Variables de configuration ──────────────────────────
RG=nyc-taxi-rg
LOCATION=eastus
CLUSTER=nyc-taxi-hdinsight
STORAGE=nyctaxistorage$(date +%s)

# ─── Creer le groupe de ressources ───────────────────────
az group create --name $RG --location $LOCATION

# ─── Creer le compte de stockage Azure Data Lake Gen2 ────
az storage account create \
  --name $STORAGE \
  --resource-group $RG \
  --location $LOCATION \
  --sku Standard_LRS \
  --kind StorageV2 \
  --hierarchical-namespace true   # Active ADLS Gen2 (compatible HDFS)

# Creer le container (equivalent d'un bucket HDFS)
az storage container create \
```

```
  --name nyc-taxi-data \
  --account-name $STORAGE

# ─── Creer le cluster HDInsight (Hadoop + Spark) ──────────────
az hdinsight create \
  --name $CLUSTER \
  --resource-group $RG \
  --location $LOCATION \
  --type Spark \
  --component-version Spark=3.1 \
  --http-password 'P@ssw0rd2024!' \
  --http-user admin \
  --ssh-password 'P@ssw0rd2024!' \
  --ssh-user sshuser \
  --storage-account $STORAGE \
  --storage-account-key $(az storage account keys list \
      --account-name $STORAGE --query '[0].value' -o tsv) \
  --storage-default-container nyc-taxi-data \
  --workernode-count 2 \
  --workernode-size Standard_D4_v2

# Recuperer l'URL SSH du cluster
az hdinsight show --name $CLUSTER --resource-group $RG \
  --query 'properties.connectivityEndpoints[?name==`SSH`].location' -o tsv
# -> nyc-taxi-hdinsight-ssh.azurehdinsight.net
```

9.3.4 Workflow CD — Option Azure HDInsight

```
# .github/workflows/cd-azure.yml
name: CD — Deploiement Azure HDInsight + ADLS2

on:
  push:
    branches: [ main ]

jobs:
  deploy-azure:
    name: Deploy to Azure HDInsight
    runs-on: ubuntu-latest
    environment:
      name: production-azure
      url: https://${{ secrets.HDINSIGHT_CLUSTER_NAME }}.azurehdinsight.net

    steps:
      - uses: actions/checkout@v4

      # ── 1. Authentification Azure ──────────────────────────
      - name: Azure Login
        uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      # ── 2. Upload du code dans ADLS2 ───────────────────────
      - name: Upload code to Azure Blob Storage
        run: |
          az storage blob upload-batch \
            --account-name ${{ secrets.AZURE_STORAGE_ACCOUNT }} \
            --account-key  ${{ secrets.AZURE_STORAGE_KEY }} \
            --source ./src \
            --destination 'nyc-taxi-data/code/src'
          az storage blob upload-batch \
            --account-name ${{ secrets.AZURE_STORAGE_ACCOUNT }} \
            --account-key  ${{ secrets.AZURE_STORAGE_KEY }} \
            --source ./jobs \
```

```
            --destination 'nyc-taxi-data/code/jobs'

      # ── 3. Soumission du job Spark via SSH sur HDInsight ──────
      - name: Submit Spark Job via SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host:     ${{ secrets.HDINSIGHT_CLUSTER_NAME }}-ssh.azurehdinsight.net
          username: ${{ secrets.HDINSIGHT_SSH_USER }}
          key:      ${{ secrets.HDINSIGHT_SSH_KEY }}
          script: |
            set -e
            STORAGE=${{ secrets.AZURE_STORAGE_ACCOUNT }}
            ADLS=abfss://nyc-taxi-data@${STORAGE}.dfs.core.windows.net

            # Installer les dependances Python sur le cluster
            pip install pyspark==3.5.0 pyarrow==14.0.1 pandas==2.1.0

            # Soumettre le job Spark
            spark-submit \
              --master yarn \
              --deploy-mode cluster \
              --driver-memory 4g \
              --executor-memory 4g \
              --num-executors 4 \
              --conf spark.hadoop.fs.azure.account.key.${STORAGE}.dfs.core.windows.net=\
                    ${{ secrets.AZURE_STORAGE_KEY }} \
              wasbs://nyc-taxi-
data@${STORAGE}.blob.core.windows.net/code/jobs/run_pipeline.py \
              --input  ${ADLS}/data/raw \
              --output ${ADLS}/data/processed \
              --models ${ADLS}/models

      # ── 4. Verification des resultats dans ADLS2 ──────────────
      - name: Verify output in ADLS2
        run: |
          az storage blob list \
            --account-name ${{ secrets.AZURE_STORAGE_ACCOUNT }} \
            --account-key  ${{ secrets.AZURE_STORAGE_KEY }} \
            --container-name nyc-taxi-data \
            --prefix 'data/processed/predictions/' \
            --query '[].name' -o tsv | head -5
          echo 'Predictions disponibles dans ADLS2'
```

9.4 Comparaison SSH vs Azure

Critere	Option A — SSH (VM Linux)	Option B — Azure HDInsight
Cout	VM : 20-80 EUR/mois	Cluster : 0.30-2 EUR/heure (on-demand)
Scalabilite	Manuelle (redimensionner la VM)	Auto-scaling integre
Maintenance	Totale (OS, securite, Hadoop)	Geree par Microsoft
Controle	Total	Limite aux parametres HDInsight
Stockage HDFS	Local sur la VM	ADLS Gen2 (illimite, HA)
Facilite CI/CD	Moyenne (config SSH manuelle)	Elevee (SDK Azure Actions)
Recommande pour	Apprentissage, on-premise	Production, equipes, enterprise

9.5 Questions — Deploiement

Question 9.1 — Option SSH — configuration et deploiement

a) Generez une paire de cles SSH ED25519 dediee au deploiement. Expliquez pourquoi ne pas reutiliser votre cle personnelle.

b) Listez et decrivez tous les secrets GitHub necessaires pour l'option SSH (nom, valeur, comment l'obtenir).

c) Dans le workflow CD SSH, que fait 'docker exec namenode hdfs dfs -ls /data/raw/' ? Que verifie-t-on ?

d) Implementez un rollback automatique : si le health check echoue apres deploiement SSH, redeployez l'image precedente.

Question 9.2 — Option Azure — provisionnement et deploiement

a) Quelle commande Azure CLI cree un Service Principal et genere le JSON pour AZURE_CREDENTIALS ?

b) Quelle est la difference entre Azure Blob Storage (WASB) et Azure Data Lake Gen2 (ABFSS) ? Lequel est preferable pour Spark ?

c) Dans le workflow CD Azure, le job Spark est soumis en '--deploy-mode cluster' et non 'client'. Expliquez la difference.

d) Pourquoi le cluster HDInsight est-il arrete entre les deploiements (az hdinsight resize) pour reduire les couts ?

Partie 10 — Monitoring et Synthese

10.1 Monitoring HDFS

```
# Sante du cluster HDFS
hdfs dfsadmin -report
# -> Affiche : Live datanodes, espace total, espace utilise

# Verifier les blocs corrompus
hdfs fsck / -list-corruptfileblocks

# Espace occupe par dossier
hdfs dfs -du -s -h /data/raw/
hdfs dfs -du -s -h /data/processed/
hdfs dfs -du -s -h /models/

# Activer le mode Safe Mode (lecture seule) manuellement
hdfs dfsadmin -safemode enter
hdfs dfsadmin -safemode leave
```

10.2 Questions — Monitoring et synthese

Question 10.1 — Spark UI et performances

a) Combien de stages genere le job complet (lecture HDFS + clean + features + RF train) ? Identifiez les shuffles.

b) Comparez le temps de lecture depuis HDFS local (Docker) vs ADLS2 Azure. Quels facteurs expliquent la difference ?

c) Apres un df.cache() sur le DataFrame nettoye, quel est l'impact sur les runs suivants dans le meme job ?

d) Expliquez Adaptive Query Execution (AQE). Testez avec enabled=true et enabled=false. Mesurez le delta.

Question 10.2 — Questions theoriques finales

a) Expliquez l'architecture HDFS : NameNode (metadonnees), DataNode (blocs), Secondary NameNode (checkpoints).

b) Qu'est-ce que le rack awareness dans HDFS ? Comment influence-t-il la replication des blocs ?

c) Comparez HDFS local (Docker) et Azure ADLS Gen2 en termes de : disponibilite, durabilite, cout, latence.

d) Proposez une architecture de production complete avec : Azure HDInsight, ADLS2, MLflow (tracking), Apache Airflow (orchestration).

Bonne chance !

Documentation : spark.apache.org | hadoop.apache.org | docs.microsoft.com/azure | docs.github.com